EDA Design Flow Integration

Lecture 7: Xilinx FPGA Flow – Implementation & Analysis

辛艾曼 Aman Sinha

Institute of Integrated Circuit Design

National Sun Yat-Sen University

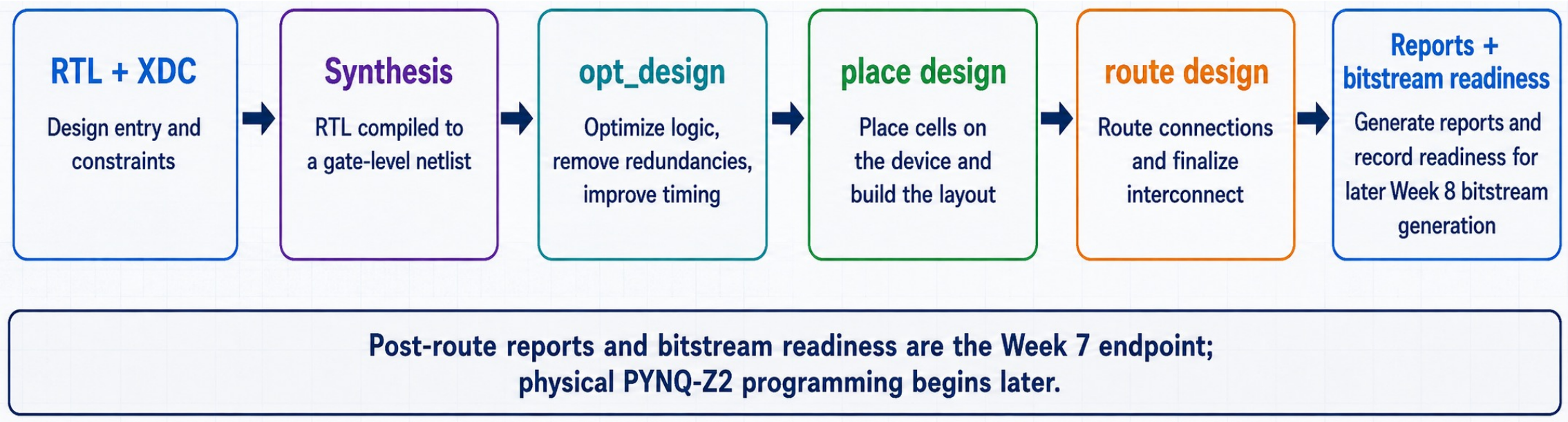
國立中山大學
National Sun Yat-sen University

FIR Implementation Studio

- **Goal:** implement the same verified three-tap FIR used in Week 6
- **Verify** simulation PASS, synthesis checkpoint, and clock constraints
- **Optimize** the synthesized design using opt_design
- **Place** FPGA logic resources using place_design
- **Route** the design using route_design
- **Inspect** post-route timing, utilization, power, DRC, and methodology reports
- **Explain** the evidence, not merely whether commands completed
- **Boundary:** no .bit generation or physical board programming in Week 7

Implementation Flow Overview

- Read RTL and constraints
- Run synthesis to build the initial netlist
- Optimize the netlist and implementation structure
- Place logic resources onto the FPGA fabric
- Route signals, generate post-route reports, and record PYNQ-Z2 bitstream readiness

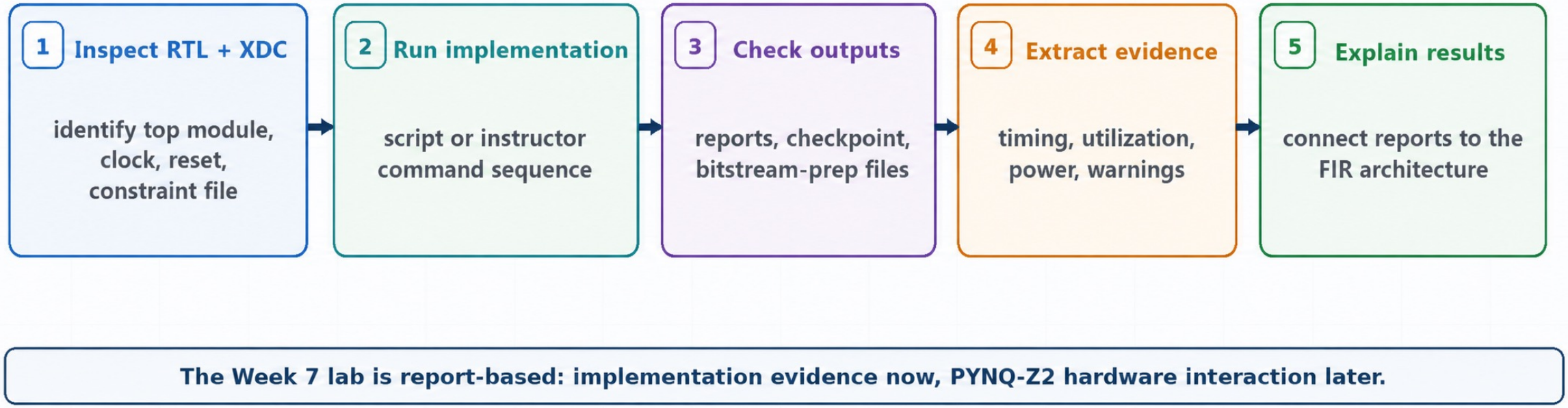


Predict Before You Route

- Discuss the following questions before examining Vivado output
  1. Predict whether synthesis and post-route WNS will be identical
  2. Explain how longer routed nets could influence critical-path delay
  3. Identify which FIR values must be held in flip-flops
  4. Consider whether FIR coefficients 1, 2, 1 require dedicated DSP blocks
  5. Distinguish a possible architectural trend from an observed report result
  6. Record predictions and compare them with actual Vivado reports

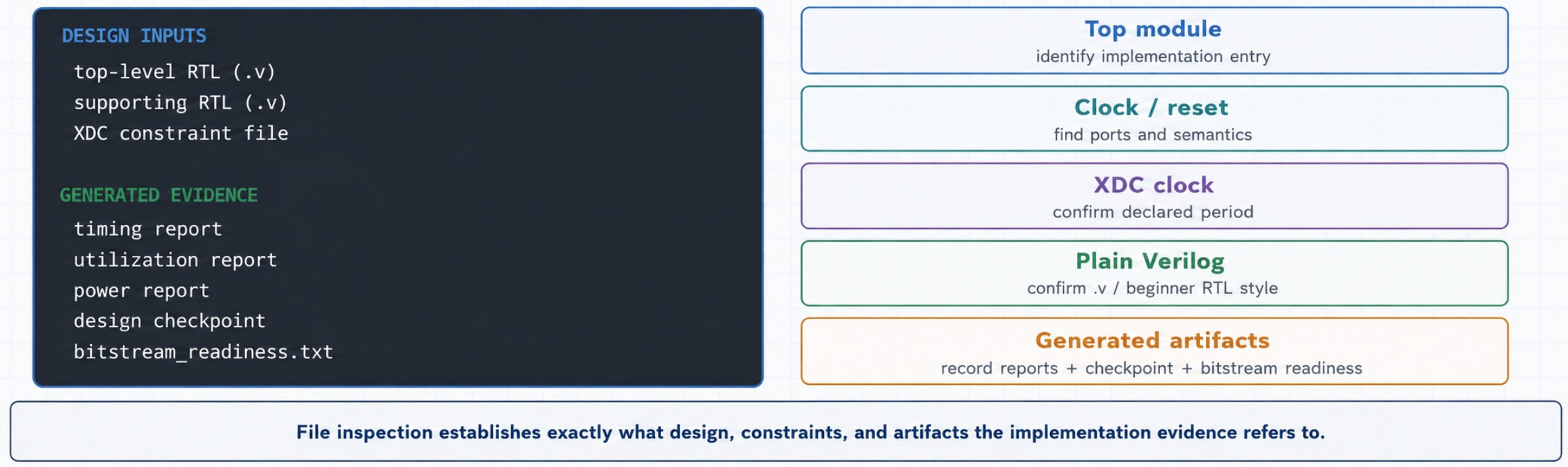
Week 7 Lab Workflow

- Inspect the provided RTL and XDC files
- Run the Vivado implementation and bitstream-preparation script or instructor command sequence
- Check that implementation reports are generated
- Extract selected timing, utilization, power, warning, and bitstream-preparation evidence
- Write a short explanation connecting reports to the FIR architecture



Part A: File Inspection

1. Identify the top module used for implementation
2. Find the clock and reset ports
3. Locate the XDC clock constraint
4. Confirm that the design uses plain Verilog
5. Record implementation reports, checkpoints, and bitstream-preparation files generated by the run



Reuse the Week 6 FIR Package

- **RTL:** rtl/fir3_pipe2.v
- **Top module:** fir3_pipe2
- **Constraints:** constraints/fir3_pipe2.xdc
- **Target FPGA:** xc7z020clg400-1 (PYNQ-Z2 device)
- **Synthesis DCP:** reports/A/post_synth.dcp
- Check the 10.000 ns clock and XSim PASS evidence
- Reopen the existing checkpoint rather than synthesizing a new design
- Keep RTL, XDC, checkpoint, and report filenames unchanged throughout the lab

Open the Synthesis Checkpoint

1. Create the output directory: file mkdir reports/week7
2. Load the synthesis result: open_checkpoint reports/A/post_synth.dcp
3. Query the clock constraints: get_clocks
4. Check the clock PERIOD property before interpreting timing results
5. Run opt_design
6. Run place_design
7. Run route_design
8. Inspect Vivado messages and resolve any failed step before proceeding
9. Note: these commands do not program a PYNQ-Z2 board

Export the Post-Route Reports

- **Timing:**
  - report_timing_summary -report_unconstrained -file reports/week7/timing.rpt
  - check_timing -verbose -file reports/week7/timing_checks.rpt
- **Resources:** report_utilization -file reports/week7/util.rpt
- **Power:** report_power -file reports/week7/power.rpt
- **Design rules:** report_drc -file reports/week7/drc.rpt
- **Methodology:** report_methodology -file reports/week7/methodology.rpt
- **Paths:** report_timing -delay_type max -max_paths 5 -file reports/week7/paths.rpt
- **Checkpoint:** write_checkpoint -force reports/week7/post_route.dcp
- Verify that the files exist and were generated after route_design
- Remember that these files contain software-estimated evidence

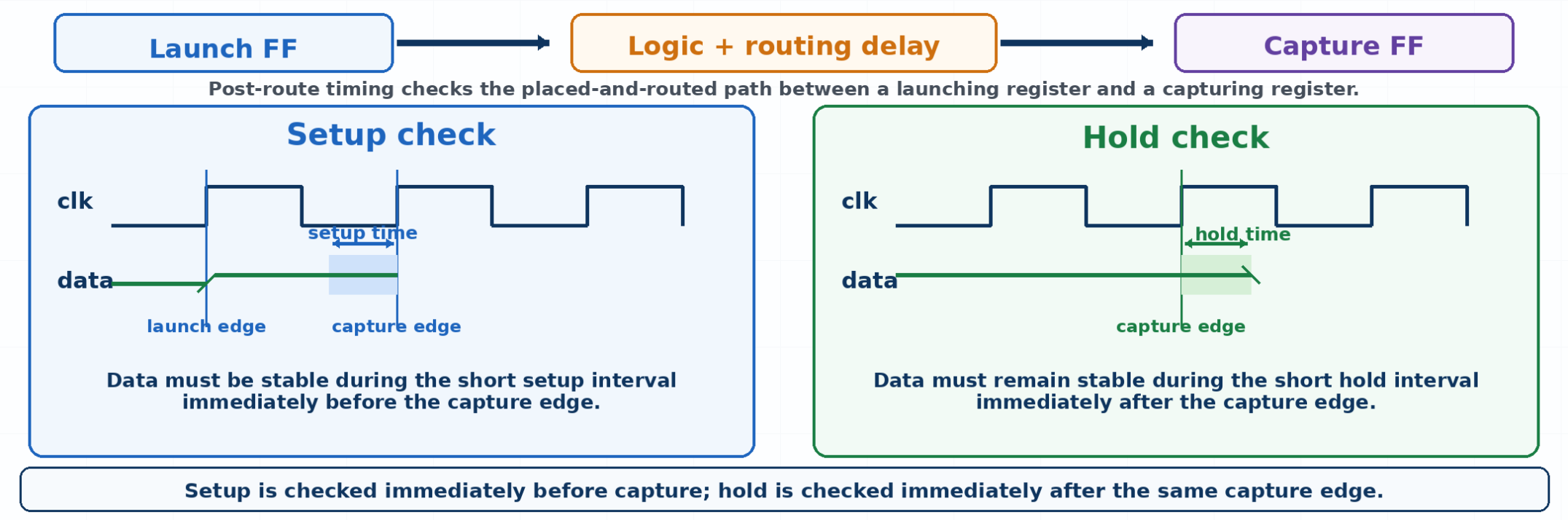
Run the Week 7 Tcl Flow

1. Start from the root of the Week 6 FIR laboratory package
2. Copy week7_impl.tcl to scripts/vivado/week7_impl.tcl
3. Create the output directories: mkdir -p logs reports/week7
4. Execute vivado -mode batch -source scripts/vivado/week7_impl.tcl
5. Or run the supplied run_week7_impl.sh after placing it in the package root
6. Inspect logs/fir3_pipe2_week7_impl.log if the tool stops or reports warnings
7. Confirm that reports/week7/post_route.dcp was generated
8. Avoid claiming timing closure until timing, constraints, and DRC are reviewed

```
vivado -mode batch \
  -log logs/fir3_pipe2_week7_impl.log \
  -journal logs/fir3_pipe2_week7_impl.jou \
  -source scripts/vivado/week7_impl.tcl
```

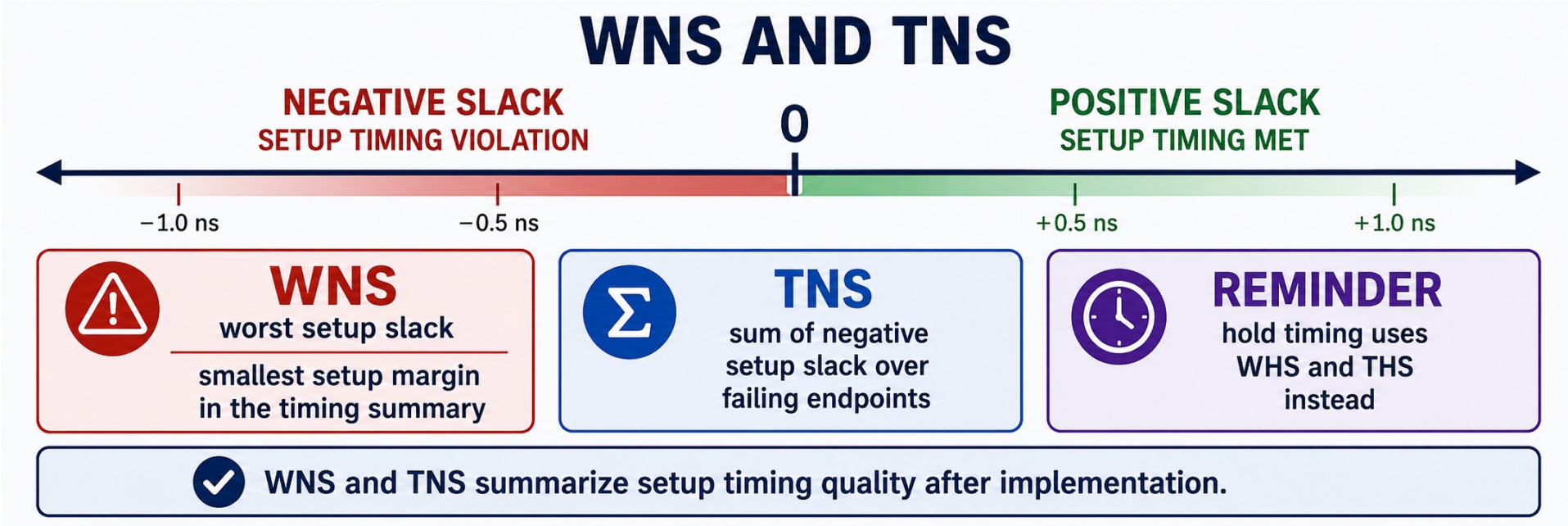
Timing After Implementation

- Implementation timing uses placed-and-routed path delays
- Setup timing checks whether data arrives before the next capture edge
- Hold timing checks whether data remains stable long enough after the capture edge
- The timing summary tells whether the requested clock target is met
- Students should report both the PYNQ-Z2 target result and the evidence behind it



WNS and TNS

- WNS means worst negative slack
- TNS means total negative slack
- Positive or zero WNS usually indicates setup timing is met
- Negative WNS means at least one path violates the timing requirement
- TNS helps indicate how widespread the setup violations are



Part B: Timing Evidence

1. Find the requested clock period
2. Find WNS/TNS and WHS/THS in the timing summary; record setup and hold status
3. Record whether timing is met
4. Identify the worst or most important path if visible
5. Explain the result in one or two technical sentences

timing_summary.rpt — extract key fields	
Requested clock period	10.000 ns
Setup status	<MET / NOT MET>
WNS	<record value>
TNS	<record value>
Hold status	<MET / NOT MET>
WHS	<record value>
THS	<record value>
Worst / limiting path	<identify if visible>

What to record

- Confirm the requested clock period
- Find WNS and TNS for setup timing
- Find WHS and THS for hold timing
- Record both setup and hold status
- Record whether the requested timing target is met
- Identify the worst or most important path if visible
- Explain the result in one or two technical sentences

Extract the relevant setup and hold evidence; do not paste a full timing report without interpretation.

Practice 1: Diagnose Timing

- Assume a requested clock period of 10 ns for these illustrative cases
  - **Case A:** WNS +0.25 ns; TNS 0 ns; WHS +0.07 ns; THS 0 ns
  - **Case B:** WNS −0.31 ns; TNS −1.40 ns; WHS +0.02 ns; THS 0 ns

1. Decide whether the setup constraint is met in each case
2. Determine whether the reported hold constraint is met in each case
3. Explain which case requires setup-path investigation
4. State why negative WNS is not a pass even if hold timing passes

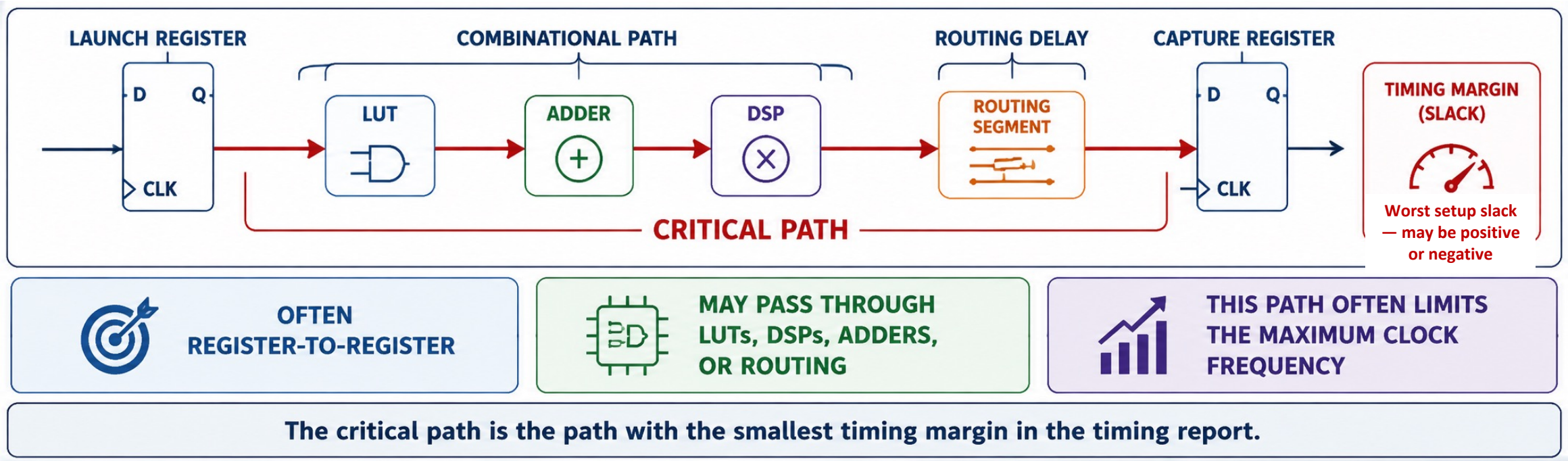
- **Warning:** these are examples, not actual Week 7 FIR results

Metric	Case A	Case B
Clock period (ns)	10	10
WNS (ns)	0.25	−0.31
TNS (ns)	0	−1.40
WHS (ns)	0.07	0.02
THS (ns)	0	0
Setup PASS/FAIL	_____	_____
Hold PASS/FAIL	_____	_____
Requires investigation?	_____	_____

Students determine the setup and hold status.
Values are illustrative, not actual FIR results.

Critical Path

- The critical path is the path with the smallest timing margin
- It often starts at a register and ends at another register
- The path may pass through LUTs, DSPs, adders, or routing
- Understanding the critical path helps explain why the clock target is or is not met
- Week 7 asks students to describe the path in simple words



Practice 2: Explain the Critical Path

1. Open reports/week7/paths.rpt after routing completes
2. Identify the launch register or startpoint of the worst setup path
3. Identify the capture register or endpoint of that path
4. Find the path slack and its required timing constraint
5. Inspect mapped cells, logic depth, and the routed interconnect
6. Compare cell delay with net delay when both appear in the report
7. Explain the limiting path in one or two technical sentences
8. Avoid guessing the physical path from RTL or synthesis estimates alone

Timing-path parameter	Value from Vivado
Startpoint (launch register)	
Endpoint (capture register)	
Clock period (ns)	
Path slack (ns)	
Cell delay (ns)	
Net/routing delay (ns)	
Dominant delay source	
Limiting logic/resources	

Source: `reports/week7/paths.rpt`. Record the actual startpoint, endpoint, path delays, and slack. Do not infer the critical path from RTL alone.

Utilization After Implementation

- Utilization reports show how many FPGA resources are used
- LUTs often implement combinational logic
- FFs implement clocked state
- DSP blocks may implement multipliers or arithmetic
- BRAMs may implement larger memories when inferred or instantiated

LUT
logic functions

FF
registers / state

DSP
multiply / arithmetic

BRAM
memory blocks

Illustrative report view

	Used	Available	Utilization
LUT	xxx	yyyy	xx%
FF	xxx	yyyy	xx%
DSP	xxx	yyyy	xx%
BRAM	xxx	yyyy	xx%

Example only — replace with your actual Week 7 report values.

How to interpret

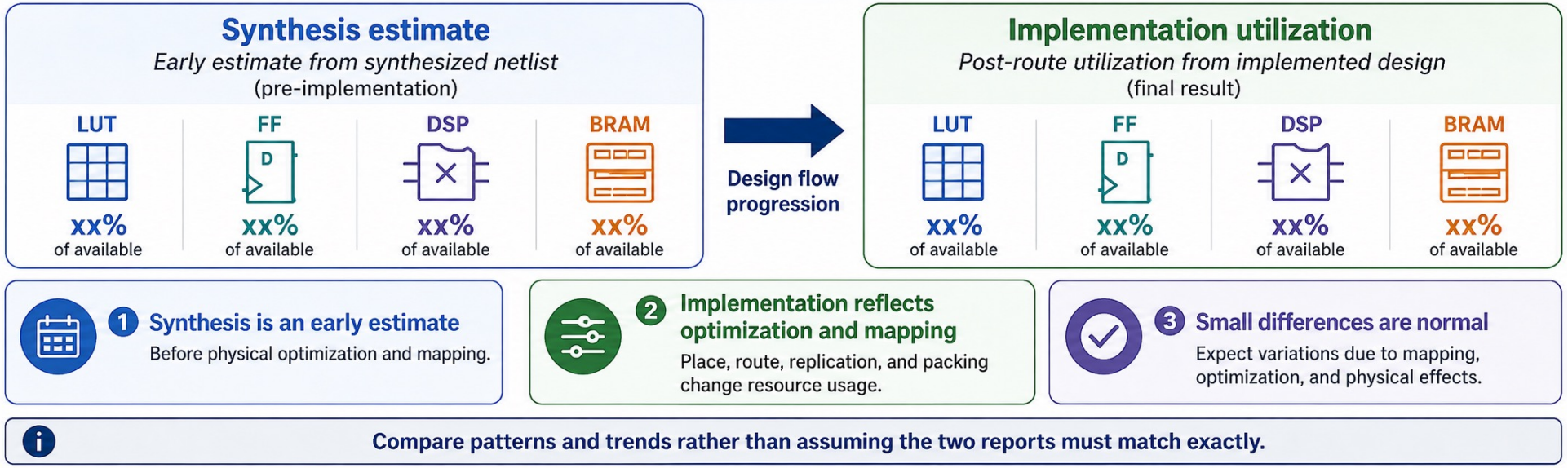
- LUT reflects logic structure
- FF grows with pipelining and state
- DSP often comes from multipliers
- BRAM appears when memory blocks are used

Implementation reports give post-place-and-route resource usage.

Do not invent utilization values in class notes; use actual report data in the lab.

Synthesis vs Implementation Utilization

- Synthesis utilization is an early estimate
- Implementation utilization reflects the implemented design after optimization and mapping
- Small differences between reports are normal
- Large differences should be investigated
- Students should cite the report name when reporting numbers



Part C: Resource Evidence

- Record LUT, FF, DSP, and BRAM usage
- Compare the resource pattern with the FIR architecture
- Explain why multipliers may affect DSP usage
- Explain why pipelining may affect FF usage
- Avoid copying a full report without interpretation

Conceptual tap arithmetic; pipeline registers not shown

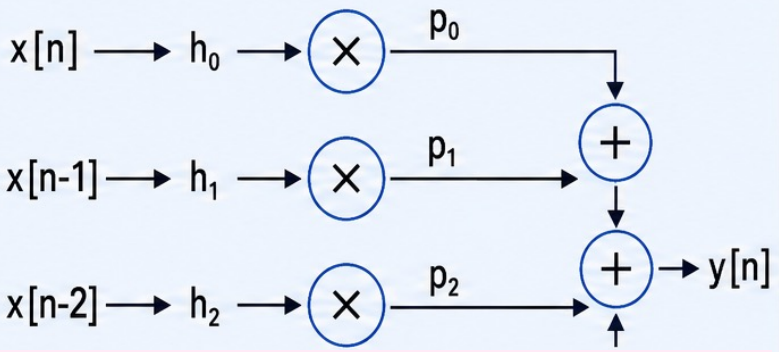
LUT
logic functions

FF
pipeline / state

DSP
multiplier / arithmetic

BRAM
dedicated memory blocks

FIR architecture connection



For FIR coefficients [1, 2, 1], constant multiplications may require no DSPs; product and output pipeline registers contribute FF usage

Interpretation checklist

- Record LUT, FF, DSP, and BRAM usage
- Compare the pattern with the FIR architecture
- Explain DSP use from multipliers
- Explain FF use from pipeline registers

Avoid copying a full utilization report without interpretation; connect the resource pattern to the architecture.

Practice 3: Compare the FIR Stages

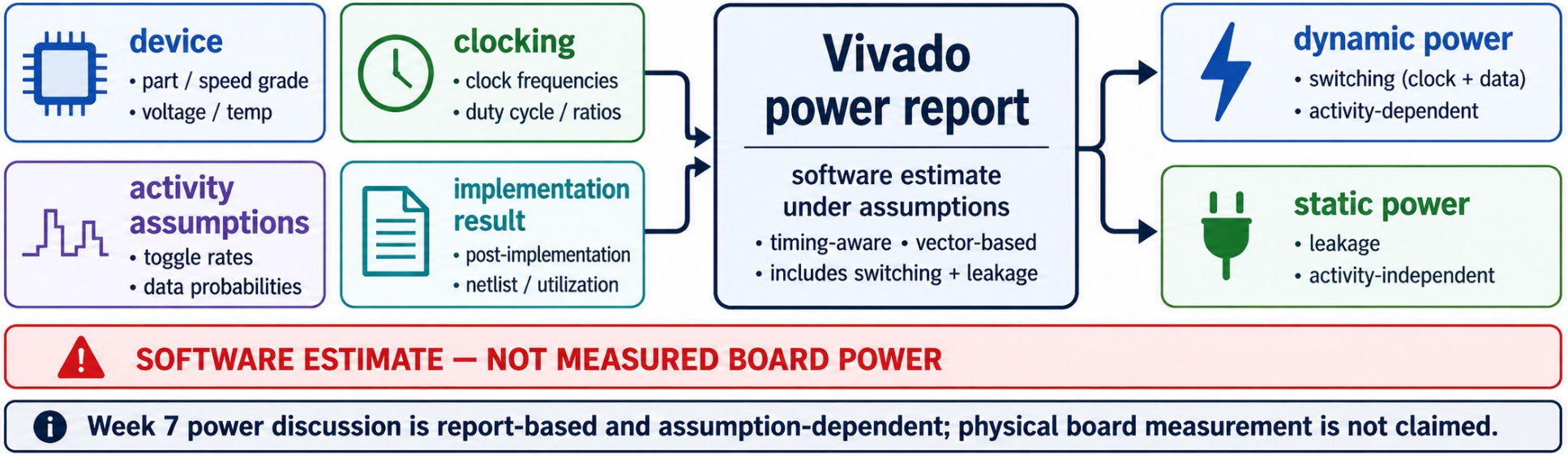
1. Open the Week 6 post-synthesis utilization and timing summaries
2. Open the Week 7 post-route utilization and timing summaries
3. Record the clock period and setup status for both stages
4. Compare LUT and flip-flop counts reported at the two stages
5. Compare DSP and block-RAM counts; do not assume either is nonzero
6. Investigate any significant count changes after implementation
7. Explain why routed net delay can alter the timing estimate
8. Cite the report filename for every measured or estimated value

Metric	Week 6 Synthesis	Week 7 Post-route
Clock period (ns)	10.000	10.000
WNS (ns)		
TNS (ns)		
WHS (ns)		
THS (ns)		
LUT count		
FF count		
DSP count		
BRAM count		
Setup/hold status		

Use Week 6 post-synthesis and Week 7 post-route reports. Record any unavailable metric as N/A, not zero. Timing values are software estimates, not board measurements.

Power Estimates

- Vivado power reports are software estimates, not measured PYNQ-Z2 board power
- Power depends on device, clocking, activity assumptions, and implementation
- Dynamic power is related to switching activity
- Static power is related to device leakage assumptions
- Use power estimates for comparison, not as exact measurement claims



Practice 4: Interpret Power

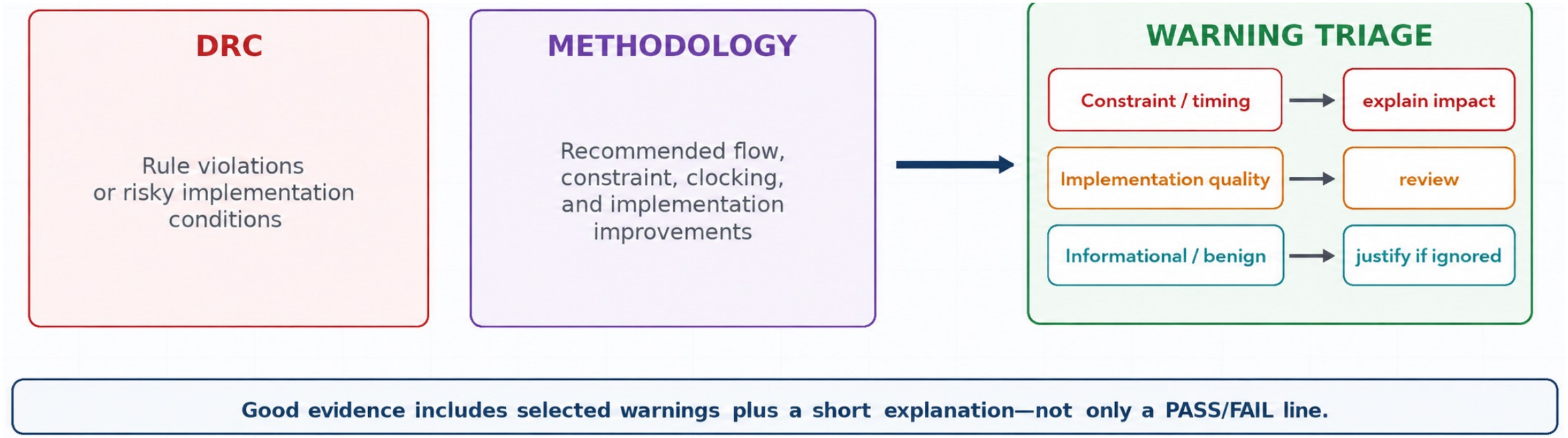
1. Open reports/week7/power.rpt from the routed design
2. Locate the estimated total on-chip power, if the report provides it
3. Separate reported dynamic power and static power
4. Identify clock frequencies and switching-activity assumptions
5. Explain why missing activity data limits accuracy
6. Write “Vivado estimates ... under the stated assumptions”
7. Do not claim measured board power or FPGA hardware validation
8. Report only values taken from the actual generated output

Parameter	Reported value / assumption
Total on-chip power (W)	
Dynamic power (W)	
Static power (W)	
Analyzed clock frequency (MHz)	
Switching-activity source	
Activity assumptions / limitations	
Estimate confidence / quality	

Source: `reports/week7/power.rpt`.
Use only available report values.
Vivado power estimates are not measured PYNQ-Z2 board power.

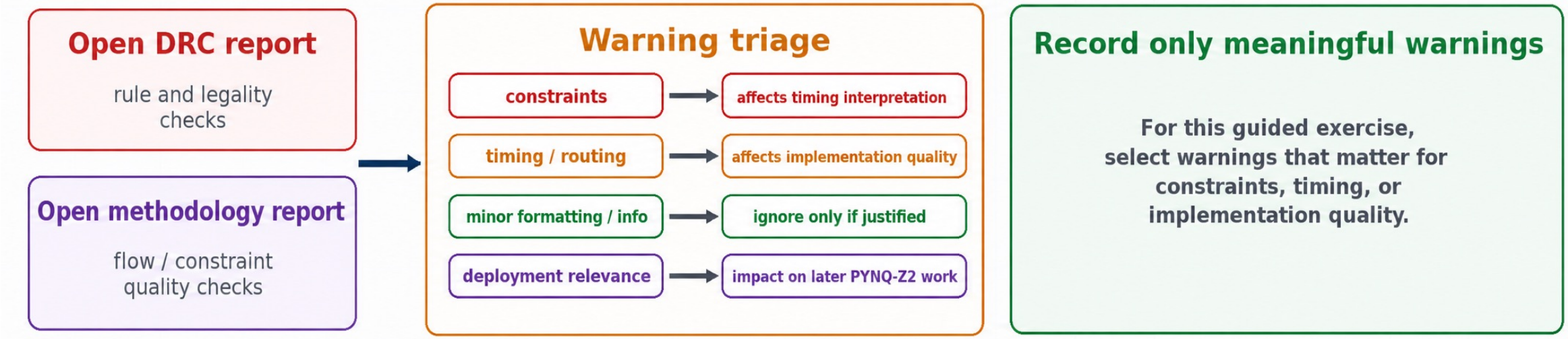
DRC and Methodology Reports

- DRC reports identify rule violations or risky implementation conditions
- Methodology reports identify recommended flow or constraint improvements
- Some warnings are harmless in a guided exercise; others must be explained
- Students should not report only PASS/FAIL lines
- Good evidence includes selected warnings and how they were interpreted



Part D: Warning Evidence

1. Open DRC and methodology reports
2. Identify warnings that matter for this guided exercise
3. Ignore minor formatting or informational messages only when justified
4. Record warnings that affect constraints, timing, or implementation quality
5. State whether each warning affects later PYNQ-Z2 deployment preparation



A warning is not automatically fatal, but it must be reviewed and justified if it could affect the implementation result.

Practice 5: Triage Warnings

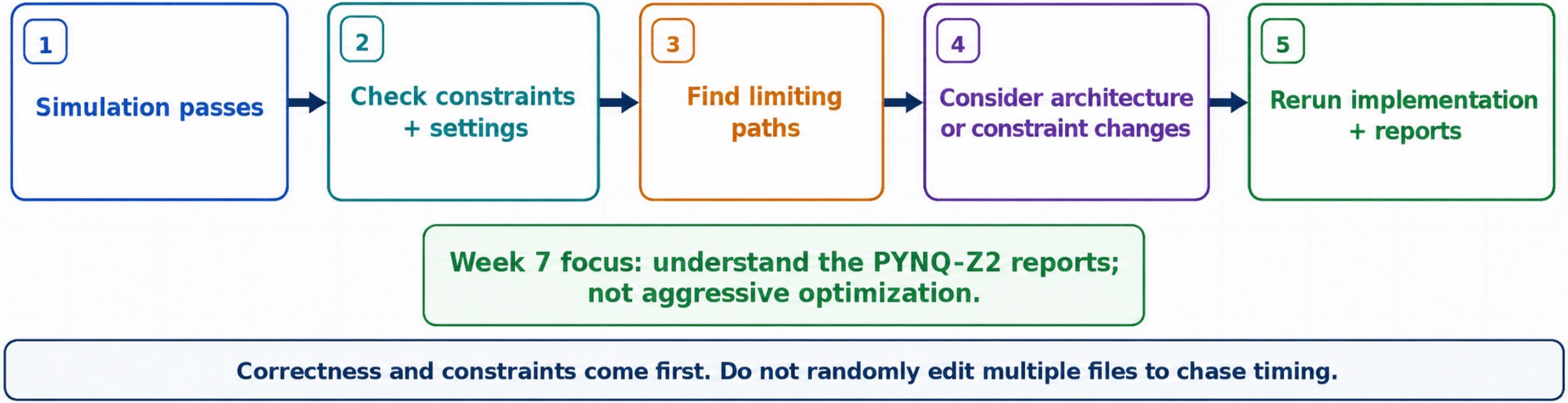
1. Open reports/week7/drc.rpt and reports/week7/methodology.rpt
2. Check for unclocked or unconstrained paths before trusting timing status
3. Review any implementation, clocking, or routing violations
4. Separate informational notes from warnings that affect the result
5. Consider whether missing board pin assignments affect later bitstream generation
6. Record the message identifier and its significance
7. Justify any warning you decide is acceptable for this guided stage
8. Remember that a completed route is not automatic timing or DRC closure

Message ID	Category	Impact on design	Action / justification
	Constraints / timing		
	Implementation / routing		
	Board readiness / I/O		
	Other significant warning		

Sources: `reports/week7/drc.rpt`, `reports/week7/methodology.rpt`, and Vivado messages. Record actual warning IDs; do not invent warnings or mark unresolved violations as harmless.

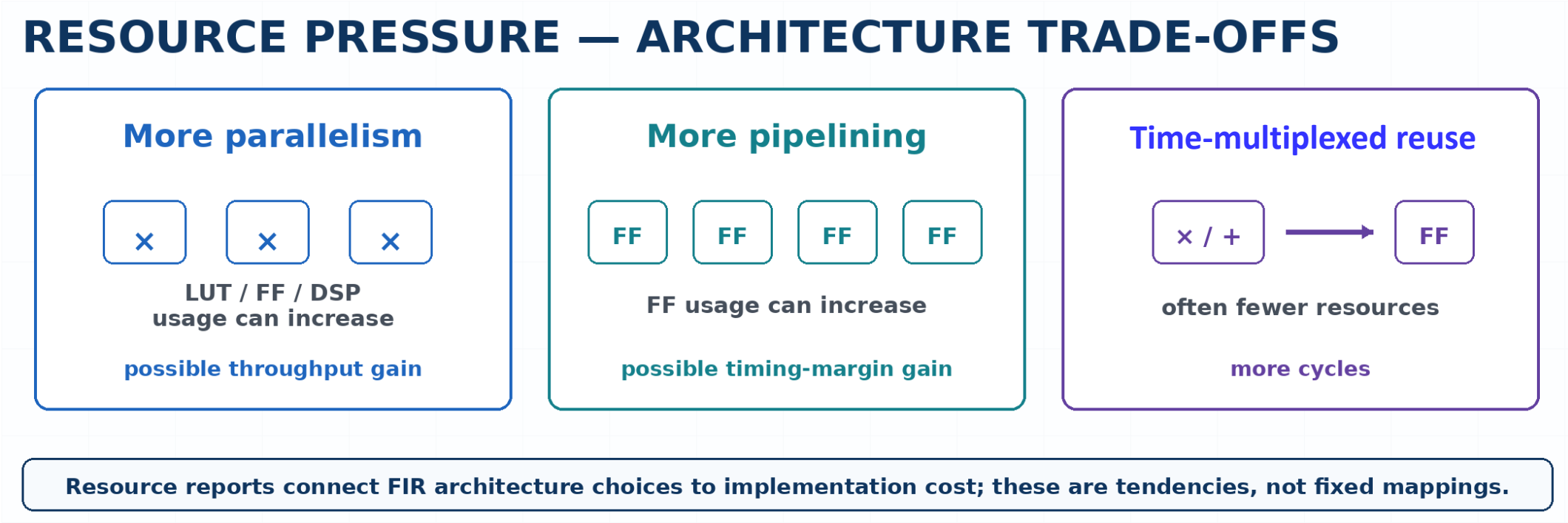
Timing Closure Workflow

1. First confirm that RTL simulation still passes
2. Then check constraints and implementation settings
3. Next identify the failing or limiting paths
4. Then consider architecture or constraint changes
5. For Week 7, focus on understanding PYNQ-Z2 reports rather than aggressive optimization



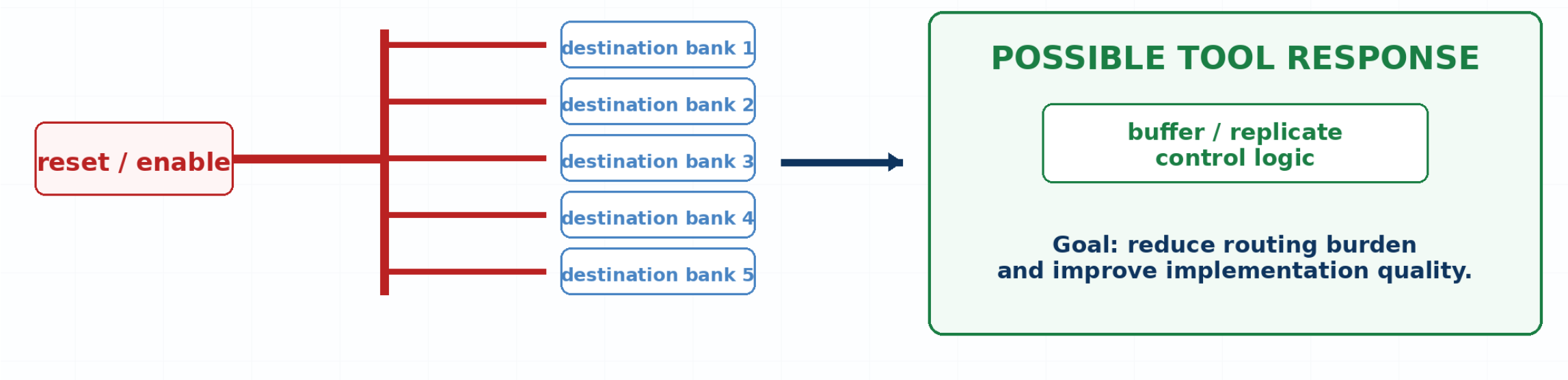
Resource Pressure

- Parallelism can increase LUT, FF, and DSP usage
- Pipelining can increase FF usage while improving timing margin
- Time-multiplexed resource reuse can reduce arithmetic hardware, but may require more cycles
- Resource reports help connect architecture choices to implementation cost
- Week 7 uses FIR versions to reinforce these trade-offs



High Fanout and Routing

- High fanout signals drive many destinations
- Reset and enable signals often have high fanout
- High fanout can affect routing and timing
- Vivado may optimize or replicate logic to handle fanout
- Students should recognize fanout messages as important evidence



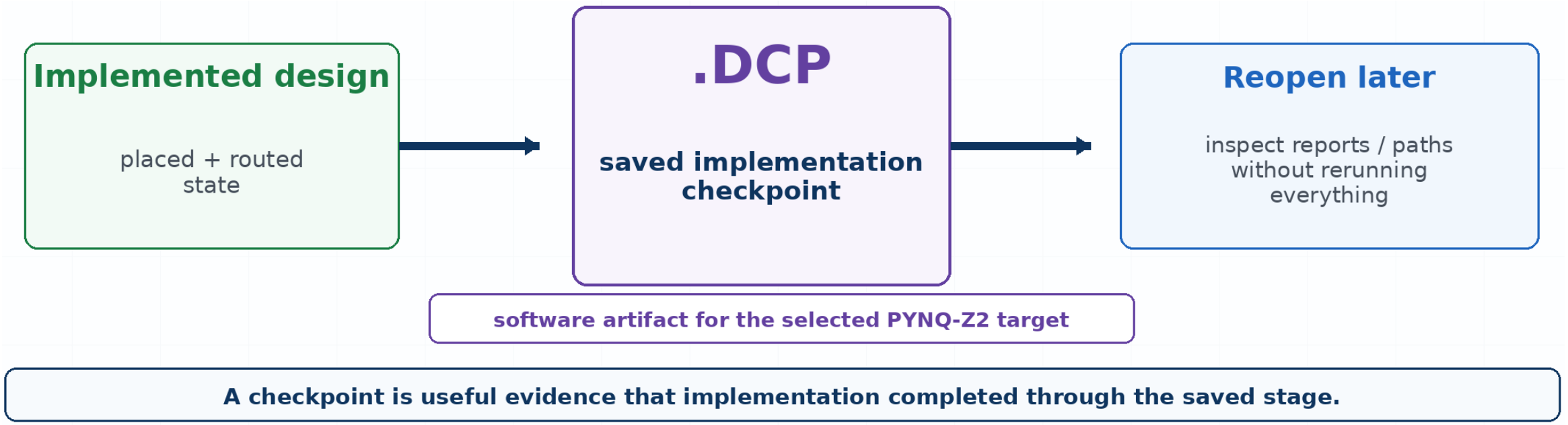
Reset and enable fanout messages are important evidence because fanout can affect routing and timing.

Choose an Evidence-Based Next Step

- **If setup fails:** first check clock constraints and timing coverage
  - Inspect the failing path before proposing an RTL or architecture change
  - Locate whether cell logic or net delay appears to dominate
  - Propose one targeted change and explain its expected effect
- **If setup passes:** check hold timing, DRC, and methodology separately
  - Preserve the routed DCP and relevant warnings and assumptions
  - Distinguish relaxing the clock target from making the hardware faster
  - Keep physical board programming and .bit generation for Week 8

Implemented Design Checkpoint

- A design checkpoint saves the implemented design state
- It can be reopened later for analysis
- It helps separate design generation from report inspection
- The checkpoint is a software artifact for the selected PYNQ-Z2 target
- It is useful evidence that implementation completed through the selected stage



Submit Evidence and Explain

1. State the target part and the clock period applied to the design
2. Record WNS/TNS and WHS/THS with setup and hold conclusions
3. Report LUT, flip-flop, DSP, and block-RAM utilization
4. Summarize power-estimation assumptions and selected DRC warnings
5. Save the routed checkpoint and the Vivado run log
6. Explain how implementation changed your understanding of the FIR
7. Answer which report supports each technical claim
8. Conclude what remains before producing a .bit or programming the PYNQ-Z2